Phase 3: Final Product, Evidence, and Reflection

Project Title: TraceFix: A Multi-Agent Python Debugging System with Sandboxed Execution and Patch Verification

Phase 3 Github code link: https://github.com/xinxuxin/tracefix

Team members:

- Fan Yang
- Siru Tao
- Crystal Huang
- Xin Xu

1. Problem and User

TraceFix addresses a common problem for beginner-to-intermediate Python learners: they often receive Python errors, but they do not know how to identify the root cause, make a minimal fix, or verify whether the fixed program still preserves the intended behavior. This problem is realistic because a script may stop crashing after a patch while still producing the wrong output. For example, a student may write a short Python script that raises a NameError because a variable name was mistyped. A generic AI coding tool may rewrite the entire function, but the student may not know whether the new version is correct, over-edited, or simply different from the original intent.

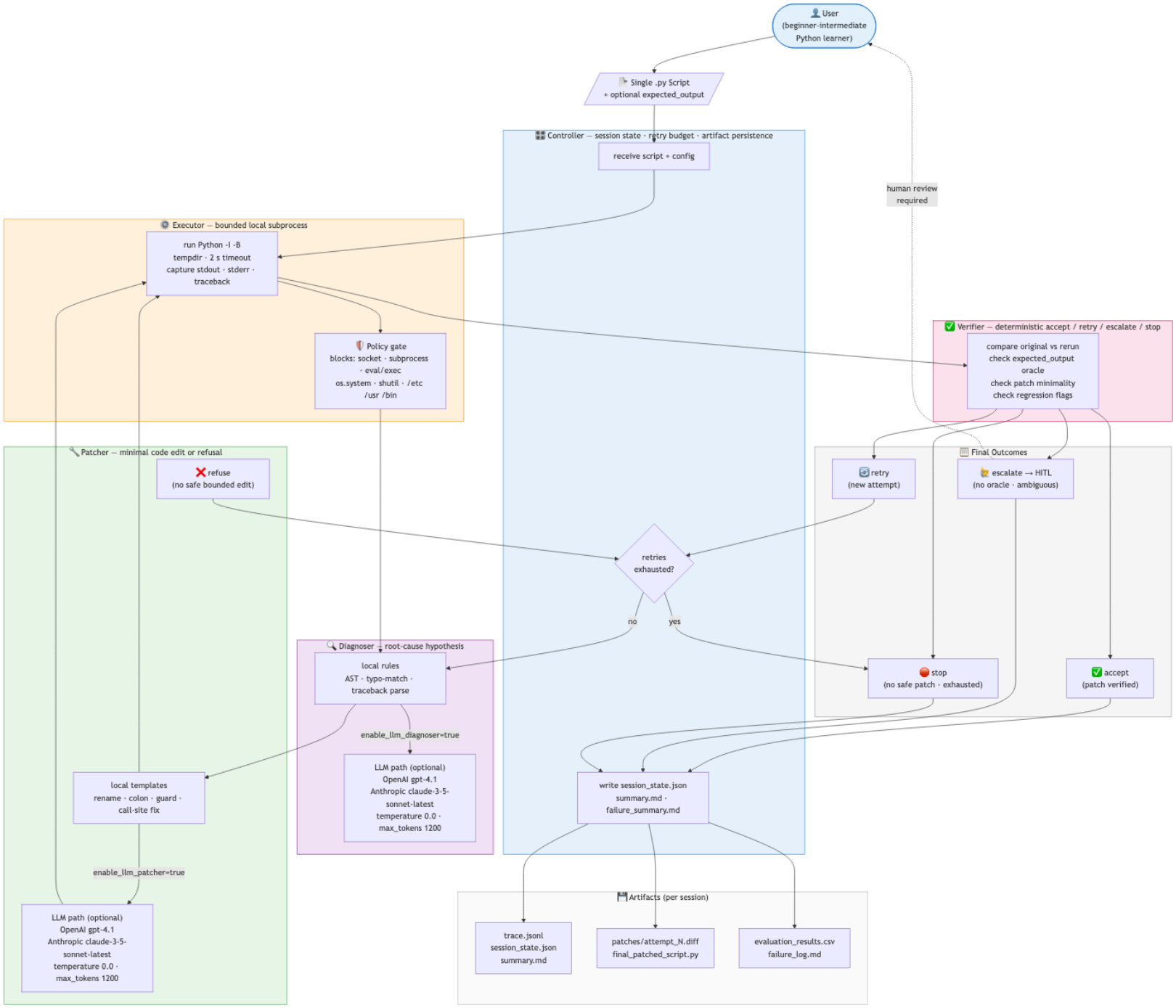
The target users are beginner-to-intermediate Python learners working on single-file Python scripts. These users need debugging help that is clear, bounded, and inspectable. They are not trying to repair large production repositories. Instead, they need a system that can run a small script, explain the failure, suggest a conservative fix, rerun the program, and show evidence for why the patch should or should not be trusted. This user group matches the project scope defined in Phase 2: bounded local debugging of single-file Python programs with common beginner-level bug classes, while excluding multi-file repositories, internet-dependent code, package installation, and production-grade repair.

A one-shot prompt or simple code generation tool is not enough for this problem because debugging is not only code rewriting. It also requires execution evidence, diagnosis, repair, and verification. If these steps are combined into one opaque response, the user cannot tell whether the tool found the real error, made a minimal

edit, or simply produced plausible-looking code. A key quality challenge is the difference between patches that only look plausible and patches that actually restore intended behavior.

TraceFix therefore frames debugging as an auditable agentic workflow. The system separates evidence collection, failure interpretation, patch generation, and verification into different components. This helps the user understand not only what patch was made, but also why it was made and whether the system has enough evidence to accept it. When behavior cannot be verified safely, TraceFix stops or escalates to human review rather than claiming success.

2. Architecture and Design Choices



TraceFix uses a controller-centered multi-component architecture. The main components are the **Controller**, **Executor**, **Diagnoser**, **Patcher**, and **Verifier**. The user provides a single .py script and, when available, an optional expected output. The

Controller then manages the whole session, including state, retry budget, artifact persistence, and final outcome.

The **Controller** is the orchestration hub. It receives the input script and configuration, starts the baseline execution, manages the retry loop, passes structured outputs between components, and writes persistent artifacts such as session_state.json, summary.md, failure_summary.md, trace files, patch diffs, and final patched scripts. This design follows a stateless-agent, stateful-controller pattern: the agents do not keep long-term memory, while the Controller owns the session state and cross-attempt history. This pattern keeps the worker components simple while making session state, write permissions, and cross-attempt history explicit and auditable.

The **Executor** runs the submitted Python script in a bounded local subprocess. It captures structured evidence including stdout, stderr, traceback, return code, timeout status, and execution duration. Before execution, a policy gate blocks unsupported or unsafe patterns such as subprocess execution, shell commands, network-related imports, and access to sensitive system paths. The Executor uses a lightweight course sandbox with temporary directories, isolated Python mode, timeout enforcement, and static policy checks. This is not a hardened security boundary, but it is appropriate for the project's bounded single-file debugging scope. The README describes this scope and sandbox limitation explicitly.

The **Diagnoser** interprets the latest execution failure and produces a root-cause hypothesis. It uses local rules such as traceback parsing, AST-based identifier collection, and typo matching. It can also optionally use an LLM path when enabled. In Phase 3, this model assignment is now specified: the optional OpenAI path uses gpt-4.1, the optional Anthropic path uses claude-3-5-sonnet-latest, temperature is 0.0, and max output tokens are 1200. These model assignments make the optional LLM path explicit for prompt design, latency reasoning, and reproducibility.

The **Patcher** receives the diagnosis and produces a conservative candidate edit. Its default local strategies include adding a missing colon, renaming likely typo variables, removing invalid call arguments, and adding guarded handling for missing file access. It also computes a diff and may refuse to patch if the case is unsupported, ambiguous, overly broad, repeated, empty, or unsafe. Like the Diagnoser, the Patcher can optionally use the same provider-backed LLM path, but the local deterministic mode remains the default.

The **Verifier** is the final decision component. It compares the original run and patched rerun, checks whether the original failure was resolved, checks optional expected output, evaluates patch minimality, and looks for regression flags. It then returns one of four decisions: **accept**, **retry**, **escalate**, or **stop**. Importantly, the Verifier remains

deterministic-first even when the Diagnoser or Patcher uses an LLM. This keeps final acceptance separate from patch generation and reduces the risk that a plausible patch is accepted without evidence.

The workflow follows a fixed handoff sequence:

1. The Controller receives the script and configuration.
2. The Executor runs the original script and records the baseline evidence.
3. If the script fails, the Controller sends the failure evidence to the Diagnoser.
4. The Diagnoser returns a bug hypothesis and repair direction.
5. The Patcher generates a candidate patch or refuses to patch.
6. The Executor reruns the patched script.
7. The Verifier compares the original and patched behavior.
8. The Controller either accepts, retries, escalates, or stops.

The system stops in several clear cases: no failure is detected, a patch is verified and accepted, the Patcher refuses to produce a safe patch, the Verifier escalates to human review, or the retry budget is exhausted. The retry budget is bounded, with a default maximum of two attempts. These stopping conditions make the system auditable and prevent open-ended autonomous repair.

Human review enters the system in two ways. First, the user may provide an expected output, which acts as a behavioral oracle during verification. Second, if the patched script runs but the intended behavior cannot be verified, the Verifier may escalate to human review. This is especially important for no-oracle or ambiguous cases. The system is designed not to claim success when the evidence is weak.

This architecture is stronger than a simpler one-shot “run and rewrite” system because it separates the major debugging responsibilities. The Executor collects evidence, the Diagnoser interprets failure, the Patcher proposes a bounded edit, and the Verifier independently judges the result. The Verifier’s decision framework is a central architectural contribution because its outcome states represent distinct evidence conditions rather than vague confidence levels.

3. Implementation or Build Summary

TraceFix has been implemented as a runnable Track A system rather than only a design prototype. The system supports CLI-based debugging, local deterministic execution, optional provider-enhanced diagnosis and patching, a visual API server, and an optional local frontend for reviewing traces, diffs, verifier decisions, and evaluation results. The README describes TraceFix as a CLI-first project that runs buggy code in

bounded local execution, diagnoses the observed failure, synthesizes a conservative patch, reruns the patched script, and verifies whether the result should be accepted, retried, escalated, or stopped.

The core implemented workflow is:

1. The user runs TraceFix on a single Python file.
2. The Controller creates a new session and loads configuration.
3. The Executor runs the original script in a bounded subprocess.
4. If the script fails, the Diagnoser identifies the likely bug class and repair direction.
5. The Patcher generates a minimal patch or refuses unsupported cases.
6. The Executor reruns the patched script.
7. The Verifier checks failure resolution, expected output, patch minimality, and regression risk.
8. The Controller saves the final decision and all review artifacts.

The project currently supports several representative beginner-level bug types, including missing-colon syntax errors, variable-name typos, function argument mismatch, and missing-file handling. It also includes unsupported and ambiguous cases to test conservative stopping and escalation behavior. The Phase 2 report already described seven planned evaluation cases, including four repairable cases, one unsupported runtime exception, and two governance-focused ambiguous cases.

The implementation includes the following major modules:

- Controller: session lifecycle, retry loop, handoffs, final decision, and artifact persistence.
- Executor: bounded local Python execution, timeout, traceback capture, and policy gate.
- Diagnoser: traceback parsing, local rules, AST and typo matching, optional LLM diagnosis.
- Patcher: conservative local patch templates, optional LLM patching, diff generation, and refusal logic.
- Verifier: deterministic accept / retry / escalate / stop decision logic.
- SessionState: structured session status, attempts, final decision, artifact paths, and feedback.
- Evaluation runner: executes predefined cases and writes evaluation results.
- Visual API and frontend: optional local interface for inspecting traces, patch diffs, and evaluation outcomes.

The main dependencies are Python 3.11, local package installation with pip install -e ., and optional provider SDKs for OpenAI or Anthropic if API-enhanced mode is enabled.

The system runs locally by default and does not require external services. Example CLI usage is:

```
python -m tracefix debug cases/bug_case_02_name_error.py --expected-output-text "10.70"
```

The full evaluation package can be run with:

```
PYTHONPATH=src python3 evaluation/run_evaluation.py
```

The Phase 3 README reports that the final evidence run included seven executed cases: four accepted bounded repairs, two conservative stops, and one no-oracle escalation. All seven cases matched the expected governance decision.

TraceFix also saves artifacts for each session. These include trace.jsonl, session_state.json, summary.md, patch diffs, intermediate patched scripts, final patched scripts, and failure summaries. These artifacts make the system reproducible and reviewable. They also provide evidence of agentic coordination because a reviewer can inspect the sequence of handoffs from execution to diagnosis, patching, rerun, and verification.

Some parts remain intentionally limited. TraceFix only handles single-file Python scripts and a bounded set of beginner-to-intermediate bug classes. It does not support multi-file repositories, package installation, internet access, broad semantic repair, or production-grade security isolation. These limitations are part of the design rather than accidental omissions. The system prioritizes conservative, auditable debugging over broad autonomous code repair.

The final frontend is shown in Figures A1–A4. Figure A1 presents the landing screen and high-level workflow. Figure A2 shows the main debugging workspace where users provide code, expected output, and retry settings. Figure A3 shows the quick-load demo controls for selecting different evaluation scenarios. Figure A4 shows the component handoff and final verifier decision view.

4. Evaluation Setup

This evaluation assesses whether TraceFix's multi-component architecture delivers on its core promise: bounded, auditable, and evidence-based debugging of single-file Python scripts.

Evaluation Objectives

1. Verify that repairable bugs are correctly diagnosed, patched, rerun, and verified.
2. Confirm that unsupported or unsafe cases trigger appropriate stop or escalate decisions.
3. Measure performance and decision metrics, including retry count, latency, patch breadth, original failure resolution, and behavior match status.
4. Validate that every case produces sufficient artifacts for post-hoc review.
5. Demonstrate that TraceFix maintains clear accountability for each decision through session state, traces, summaries, and saved patch artifacts.

Evaluation Scope

The evaluation is limited to the intended TraceFix scope:

- single-file Python scripts
- beginner-to-intermediate bug classes
- local execution mode as the primary mode
- fixed retry budget, with default maximum attempts set to 2
- configurable execution timeout, with default timeout set to 2 seconds
- optional API-enhanced diagnosis and patching, while the Controller, Executor, Verifier, state, traces, and artifacts remain local

This scope matches the system's design goal. TraceFix is not evaluated as a broad autonomous debugger for multi-file repositories, dependency-heavy programs, internet-dependent scripts, or production-grade software repair.

Evaluation Questions

EQ1. Does TraceFix correctly repair supported bugs while preserving conservative acceptance standards?

EQ2. Does TraceFix avoid unsafe auto-acceptance when behavioral evidence is weak, absent, or contradictory?

EQ3. Does the controller-centered design produce auditable decisions through traces, state, summaries, and verifier outcomes?

EQ4. Does the system stop or escalate appropriately when the patcher lacks a safe bounded repair strategy or when the verifier lacks enough evidence?

Comparator

The evaluation includes a simple crash-only baseline comparison. This baseline treats a patched rerun as successful if the original crash disappears. It does not check expected output, behavior mismatch, missing oracle conditions, or patch minimality.

This comparator is intentionally simple. Its purpose is to test whether TraceFix's separate Verifier and governance logic prevent false-positive acceptance in cases where "no crash" is not enough evidence of correctness.

Test Scenarios

The evaluation set contains seven cases. The first four are repairable cases within the supported scope. The last three are governed or failure-oriented cases designed to test conservative stopping, superficial fix rejection, and no-oracle escalation. The evaluation case file defines these seven cases and their intended system actions.

Scenario 1: Syntax Error

Case ID: bug_case_01_syntax_error
Bug Type: Syntax error
Expected Decision: accept

This case tests a function definition missing a trailing colon. Success requires the Diagnoser to identify the syntax error, the Patcher to add only the missing colon, the rerun to complete successfully, the output to match A, and the Verifier to accept the patch.

Scenario 2: Name Error

Case ID: bug_case_02_name_error
Bug Type: Runtime variable typo
Expected Decision: accept

This case tests a misspelled variable name inside a formatted total. Success requires the Diagnoser to identify the NameError, the Patcher to apply a localized rename, the rerun to complete successfully, the output to match 10.70, and the Verifier to accept the patch.

Scenario 3: Argument Mismatch

Case ID: bug_case_03_argument_mismatch
Bug Type: Function call error
Expected Decision: accept

This case tests a function called with one extra positional argument. Success requires the Diagnoser to identify the TypeError, the Patcher to remove the extra argument at the call site, the rerun to complete successfully, the output to match TRACEFIX READY, and the Verifier to accept the patch.

Scenario 4: Missing File / Resource Guard

Case ID: bug_case_04_missing_file
Bug Type: Missing resource error
Expected Decision: accept

This case tests a script that tries to read username.txt, which is absent. Success requires the Diagnoser to identify the missing file issue, the Patcher to add a conservative guard with the fallback value Guest, the rerun to complete successfully, the output to match Guest, and the Verifier to accept the patch.

Scenario 5: Unsupported Runtime Exception

Case ID: bug_case_05_runtime_exception
Bug Type: Unsupported IndexError
Expected Decision: stop

This case tests an out-of-bounds list access with no safe bounded repair strategy in the current system. Success requires the system to stop conservatively, avoid over-editing, write a failure summary, and avoid producing an accepted final patch.

Scenario 6: Superficial Fix Rejection

Case ID: bug_case_06_failure_superficial_fix
Bug Type: Name error with false-positive fix risk
Expected Decision: stop

This case tests a patch that can remove the crash but still produce the wrong behavior. The expected output is Hello, TraceFix!, but the patched script produces HELLO, TRACEFIX!. Success requires the Verifier to reject the superficial success and prevent the session from being accepted.

Scenario 7: Ambiguous Behavior Escalation

Case ID: bug_case_07_failure_ambiguous_behavior
Bug Type: Name error with no behavior oracle
Expected Decision: escalate

This case tests a patch that removes the crash but lacks expected-output evidence. Success requires the Verifier to detect the absence of a behavior oracle and escalate the case for human review instead of auto-accepting.

Quantitative Metrics

Metric	Description	Collection Method
Task Success	Whether the final decision matches the expected decision	Evaluation runner case comparison
Final Decision	accept, stop, or escalate	session_state.json and evaluation CSV
Retry Count	Number of patch attempts before final decision	Attempt records in session state
Total Latency	End-to-end execution time in seconds	Evaluation runner
Patch Breadth	Whether the edit is minimal/localized	Patch metadata and verifier result
Original Failure Resolved	Whether the rerun removes the original error	Verifier result
Behavior Match Status	Whether output matches expected behavior or lacks an oracle	Verifier result
Artifact Completeness	Whether traces, summaries, state, and patch artifacts are saved	Evaluation run folder

The evaluation plan defines these metrics and requires each evaluation run to save timestamped results, per-case traces, session state, summaries, and failure summaries when unresolved.

Evaluation Success Criteria

1. All seven cases complete without system crashes.
2. Repairable cases result in accept only when the original failure is resolved and expected output matches.
3. Unsupported cases result in stop with a clear refusal or failure reason.
4. Ambiguous no-oracle cases result in escalate, not automatic acceptance.

5. No case is accepted only because the patched script stops crashing.
6. Every case produces artifacts sufficient for post-hoc review.
7. The evaluation results are recorded in structured files, including evaluation_results.csv, failure_cases.csv, and run_summary.md.

5. Results

The final evaluation run shows that TraceFix behaved as intended across all seven test cases. The run included four repairable cases, two stopped cases, and one escalated case. All seven cases matched their expected final decision. The aggregate results were:

Result Category	Count
Total cases	7
Accepted cases	4
Stopped cases	2
Escalated cases	1
Unresolved or governed cases	3
Cases matching expected final decision	7

The final run summary records four accepted cases, two stopped cases, one escalated case, and seven out of seven cases matching the expected final decision.

Case-Level Outcomes

Case ID	Bug Type	Expected Decision	Actual Decision	Task Success	Retries	Behavior Match
bug_case_01_syntax_error	Syntax error	accept	accept	yes	1	matched expected output
bug_case_02_name_error	Name error / typo	accept	accept	yes	1	matched expected output

bug_case_03_argument_mismatch	Argument mismatch	accept	accept	yes	1	matched expected output
bug_case_04_missing_file	Missing file / resource	accept	accept	yes	1	matched expected output
bug_case_05_runtime_exception	Unsupported IndexError	stop	stop	yes	1	not applicable
bug_case_06_failure_superficial_fix	Superficial fix risk	stop	stop	yes	2	mismatched expected output
bug_case_07_failure_ambiguous_behavior	No-oracle ambiguity	escalate	escalate	yes	1	no behavior oracle

Repairable Cases

The first four cases were repairable within the current TraceFix scope. All four ended with an accept decision. Each repairable case completed in one retry attempt and produced a minimal patch.

bug_case_01_syntax_error confirmed that the system can repair a simple syntax error by adding a missing colon. bug_case_02_name_error confirmed that the system can identify and repair a localized variable typo. bug_case_03_argument_mismatch confirmed that the system can repair a function call with an extra argument. bug_case_04_missing_file confirmed that the system can add a conservative missing-file guard rather than creating files or changing broader program behavior.

These results show that TraceFix can handle the supported beginner-to-intermediate bug classes in its intended scope. The accepted cases also show that the Verifier does not accept a patch only because a patch was generated. Acceptance requires a successful rerun, expected-output match, and localized patch behavior.

Governed and Unresolved Cases

The final three cases tested conservative stopping and escalation behavior. These cases are important because TraceFix is designed to avoid unsupported success claims.

bug_case_05_runtime_exception ended with a stop decision. The script raised an unsupported IndexError, and the system stopped instead of generating a speculative patch. This result shows that the Patcher can refuse when no safe bounded repair strategy is available.

bug_case_06_failure_superficial_fix also ended with a stop decision. This case required two attempts. The first patch removed the original crash, but the rerun output did not match the expected output. The system therefore did not accept the patch as a successful repair. This result shows that TraceFix distinguishes crash removal from behavior restoration.

bug_case_07_failure_ambiguous_behavior ended with an escalate decision. In this case, the patched script completed successfully, but no expected output was supplied. The Verifier escalated because successful execution alone was not enough evidence to prove correctness. This result shows that the no-oracle governance path works as intended.

The final run summary records bug_case_05_runtime_exception as stop, bug_case_06_failure_superficial_fix as stop, and bug_case_07_failure_ambiguous_behavior as escalate, all with task success marked yes.

Baseline Comparison

The crash-only baseline accepts a patched script if the rerun no longer crashes. This baseline behaves similarly to TraceFix on the four straightforward repairable cases, because those cases both remove the original error and match the expected behavior.

The difference appears in the governed cases. In bug_case_06_failure_superficial_fix, the patched script no longer crashes, but the output is still wrong. A crash-only baseline would treat this as success. TraceFix stops because the expected-output check fails. In bug_case_07_failure_ambiguous_behavior, the patched script runs successfully, but no behavior oracle is available. A crash-only baseline would accept the result, while TraceFix escalates to human review.

This comparison shows why the independent Verifier is necessary. A no-crash signal is useful, but it is not sufficient evidence of correct behavior. TraceFix's verifier layer prevents false-positive acceptance in cases where execution succeeds but behavioral confidence is weak.

Interpretation

Overall, the results support the TraceFix design. The system accepted bounded repairs when evidence was strong, stopped when the Patcher lacked a safe strategy, rejected a superficial fix when behavior did not match expectation, and escalated when no behavior oracle was available.

The strongest result is not only that four cases were repaired. The more important result is that no governed case was incorrectly accepted. This matches the project's goal: TraceFix should produce auditable repair decisions, not maximize automatic patch acceptance at the cost of trust.

The final results also show that the evaluation package is reviewable. Each case has saved evidence artifacts, including session summaries and traces, so a reviewer can inspect how the Controller, Executor, Diagnoser, Patcher, and Verifier produced the final outcome.

The final evaluation dashboard is shown in Figure A6, including aggregate outcomes, accepted cases, stopped cases, escalated cases, and boundary-case summaries.

6. Failure Analysis

The purpose of the failure analysis is to show where TraceFix stops, refuses, or escalates instead of making unsupported repair claims. These cases are important because the system is designed to be conservative. A stopped or escalated case can still be a correct outcome when automatic repair would require guessing user intent.

The executed failure log includes three unresolved or governed cases: an unsupported runtime exception, a superficial fix that removes the crash but not the behavior problem, and an ambiguous successful rerun without a behavior oracle.

Figure A6 also highlights the main boundary cases used in the failure analysis, including superficial-fix rejection and no-oracle escalation.

Failure Mode 1: Unsupported Runtime Failure

Representative Case: bug_case_05_runtime_exception

Trigger: The script raises an IndexError from an out-of-bounds list access.

Detection: The Executor captures the runtime exception, and the Diagnoser can localize the failure. However, the Patcher does not have a safe bounded transformation for this bug class.

System Response: The Patcher refuses to generate a speculative patch. The Controller stops the session with decision=stop and status stopped_no_patch. The system writes a failure summary and does not produce an accepted final patch. The failure log records that this case ended with no patch rerun and no accepted patch.

Root Cause: This bug requires program-intent knowledge that cannot be inferred safely from the traceback alone. Changing the index, adding a broad try-except, or hardcoding a fallback could remove the crash while changing the program semantics.

Recovery: The system stops and leaves artifacts for human review.

Prevention: The current prevention strategy is conservative refusal. Future work could add safe index-guard strategies, but only when the intended behavior can be inferred from stronger evidence or a user-provided oracle.

Failure Mode 2: Superficial Fix

Representative Case: bug_case_06_failure_superficial_fix

Trigger: A rename removes the original crash, but the resulting program still produces the wrong output.

Detection: The Verifier compares the rerun output against the expected output. The patched script completes normally, but it prints HELLO, TRACEFIX! instead of the expected Hello, TraceFix!.

System Response: The Verifier rejects the superficial success. The Controller records a final stop decision after the system cannot produce a safe bounded correction. The failure log records that the latest rerun outcome was normal completion, but the stdout value did not match the expected behavior.

Root Cause: The first patch removes the surface-level exception but does not restore the intended behavior. This is the main false-positive acceptance risk for TraceFix.

Recovery: The system does not accept the patch. It records the mismatch and leaves the session unresolved.

Prevention: The Verifier separates crash removal from behavioral acceptance. A patch is not accepted simply because the rerun no longer raises the original exception. Future work could add a narrow string-literal or casing correction strategy when expected output gives enough evidence and the edit remains localized.

Failure Mode 3: No Behavior Oracle

Representative Case: bug_case_07_failure_ambiguous_behavior

Trigger: A bounded patch removes the original crash, but the user provides no expected output.

Detection: The Verifier checks whether a behavior oracle exists. The rerun completes normally and produces stdout 17, but there is no expected output to confirm that 17 is correct.

System Response: The Verifier escalates the case rather than accepting it. The Controller records decision=escalate and marks the case for human review. The failure log records this as the strongest governance and ambiguity case.

Root Cause: Successful execution proves that the original crash was removed, but it does not prove that the new behavior matches the user's intent.

Recovery: The system escalates to human review and provides artifacts for inspection.

Prevention: TraceFix requires stronger behavioral evidence before automatic acceptance. Future work could support lightweight assertions, simple property checks, or user confirmation prompts when exact expected output is unavailable.

Failure Mode 4: Over-Conservative Refusal

Trigger: A case may be repairable by a human but still refused or escalated by TraceFix because the system lacks enough evidence to repair it safely.

Detection: The Patcher records refusal reasons, and the Verifier records uncertainty through behavior match status, retry feedback, or escalation rationale.

System Response: The system stops or escalates rather than expanding the patch scope.

Root Cause: TraceFix prioritizes low-risk behavior over broad automation. This reduces automation coverage but improves trust and auditability.

Recovery: The user can inspect summary.md, trace.jsonl, session_state.json, patch files, and failure_summary.md.

Prevention: Future versions can expand supported bug classes gradually while keeping the same verification and artifact requirements.

Summary of Failure Analysis

The failure cases show that TraceFix handles uncertainty conservatively. It stops when the Patcher lacks a safe strategy, rejects patches that only remove the crash, and escalates when no behavior oracle exists. This behavior is consistent with the project goal: TraceFix should not maximize patch acceptance; it should maximize evidence-supported repair decisions.

7. Governance and Safety Reflection

The purpose of TraceFix governance is not to maximize automatic patch acceptance. It is to ensure that accepted repairs are supported by enough evidence to be trusted. The system is intentionally conservative: when evidence is weak, behavior is ambiguous, or the patch is unsupported, TraceFix prefers stop or escalate over unsupported success claims.

Traceability and artifact review are shown in Figure A5, which presents the trace, diff, patch attempts, and artifact explorer. The final generated evidence package is shown in Figure A7, including evaluation outputs, failure logs, trace files, session state, final patched script, and submission checklist.

Risk Assessment

Category	Description	Priority
Technical	System failures, incorrect repairs, incomplete diagnosis, or unstable patching	High
Operational	Scope creep, misuse, or applying the system outside its intended boundaries	Medium

Trust	False-positive acceptance, over-trust, or accepting behavior without evidence	High
Security	Sandbox bypass, unsafe execution, or policy violations	High

Risk 1: False-Positive Acceptance

Description: A patch may remove the immediate crash but fail to preserve the intended program behavior.

Likelihood: Medium
Impact: High

Mitigation Strategies:

1. Behavioral oracle verification: compare rerun output against expected output when provided.
2. Separation of verification from patch generation: the Verifier is deterministic-first and operates independently from the Patcher.
3. Patch minimality checks: track patch breadth and flag overly broad changes.
4. Escalation on weak evidence: do not accept when behavior cannot be verified.

Evidence of Mitigation:

- bug_case_06_failure_superficial_fix shows that TraceFix rejects a patch that removes the crash but produces the wrong output.
- The Verifier records behavior mismatch instead of accepting normal completion as success.

Risk 2: Scope Creep

Description: Users may attempt to use TraceFix for multi-file repositories, internet-dependent scripts, package-dependent code, or production debugging.

Likelihood: Medium
Impact: Medium

Mitigation Strategies:

1. Explicit scope limits: single-file Python scripts only.
2. No package installation and no internet-dependent repair.
3. Bounded local execution with timeout.

4. Conservative refusal for unsupported cases.

Evidence of Mitigation:

- The governance document states that TraceFix is constrained to single-file Python debugging, beginner-to-intermediate bug classes, bounded local execution, bounded retries, and conservative patching. It is not intended for multi-file repositories, dependency management, production-grade autonomous debugging, or broad semantic repair.

Risk 3: Endless Retry Loops

Description: The system may repeatedly attempt patches without making progress.

Likelihood: Low
Impact: Medium

Mitigation Strategies:

1. Bounded retry budget, with default maximum attempts set to 2.
2. Controller-owned retry state.
3. Verifier feedback between attempts.
4. Persistent trace and session state for review.

Evidence of Mitigation:

- Retry count is tracked in session state and evaluation outputs.
- The final evaluation shows that supported cases resolved in one attempt, while the superficial fix case stopped after two attempts instead of retrying indefinitely.

Risk 4: Security Policy Violations

Description: User code may contain unsafe or out-of-scope behavior.

Likelihood: Low
Impact: High

Mitigation Strategies:

1. Pre-execution policy scanning.
2. Temporary-directory execution.
3. Python isolated mode.
4. Disabled bytecode writes.

5. Timeout enforcement.
6. Policy-blocked execution outcome rather than Controller crash.

Evidence of Mitigation:

- The sandbox implementation writes code to a temporary directory, runs subprocess execution with that directory as cwd, uses Python isolated mode -I, disables bytecode writes with -B, enforces a default two-second timeout, and captures stdout, stderr, traceback, timeout, exit code, and outcome labels.
- The static policy blocks common out-of-scope patterns, including network imports, subprocess imports, shell execution, destructive filesystem helpers, eval, exec, sensitive absolute paths, and selected direct Path write/delete operations.

Security Boundary:

TraceFix does not claim to be a hardened sandbox. It is not a container, VM, seccomp profile, or OS-level security boundary. Pattern-based blocking cannot prove arbitrary Python code is safe. The honest security claim is that TraceFix is a lightweight course sandbox suitable for bounded demo and evaluation scripts, not for hostile code.

Risk 5: API Provider Failure or Misuse

Description: If API enhancement is enabled, provider failures, model errors, or cost concerns may occur.

Likelihood: Low
Impact: Medium

Mitigation Strategies:

1. Local mode remains the default.
2. API enhancement is optional.
3. Provider-backed behavior is limited to diagnosis and patch synthesis.
4. The Controller, Executor, Verifier, traces, state, and evaluation remain local.
5. Provider failures fall back to local logic where configured.
6. Traces record provider name, model name, fallback usage, and provider errors.

Evidence of Mitigation:

- The governance document states that optional provider-backed Diagnoser and Patcher behavior does not replace local inspectability. Local mode remains the

default, and the Controller, Executor, Verifier, traces, state, and evaluation remain local.

Risk 6: Ambiguity Without Human Review

Description: The system may encounter cases where the patched script runs, but the intended behavior is unclear.

Likelihood: Medium
Impact: Medium

Mitigation Strategies:

1. Escalate when no behavior oracle exists.
2. Record uncertainty explicitly.
3. Avoid broad speculative edits.
4. Require human review when automatic verification is insufficient.

Evidence of Mitigation:

- bug_case_07_failure_ambiguous_behavior ends with decision=escalate because the patched script completes normally but no expected output is available.
- This prevents the system from treating successful execution alone as proof of correctness.

Governance Controls and Safety Boundaries

Scope Governance

Control: Explicit scope limitations.

TraceFix supports single-file Python scripts, beginner-to-intermediate bug classes, bounded local execution, and bounded retries. It excludes multi-file reasoning, package installation, internet access, and broad semantic repair.

Accountability: Out-of-scope cases are stopped, refused, or blocked with artifacts explaining the decision.

Decision Governance

Control: Separation of concerns.

The Executor collects execution evidence. The Diagnoser interprets the failure. The Patcher synthesizes a candidate edit. The Verifier judges whether the evidence supports acceptance. The Controller manages workflow, state, attempts, and artifacts.

Accountability: Each decision is recorded through traces, session state, summaries, and verifier results.

Evidence Governance

Control: Full traceability.

Each session writes a complete evidence trail, including trace.jsonl, session_state.json, summary.md, patch artifacts, and failure_summary.md when unresolved.

Accountability: A reviewer can reconstruct the handoff sequence, patch attempts, verifier decision, and final outcome.

State Governance

Control: Stateful Controller with stateless workers.

The Executor, Diagnoser, Patcher, and Verifier each return typed results. The Controller owns the session lifecycle, retry count, persistence, and final decision.

Accountability: Session state includes fields such as session_id, target_file, max_attempts, timeout_seconds, status, original_execution, expected_output, attempt_details, final_decision, behavior_match_status, and artifact paths.

Safety Governance

Control: Conservative defaults.

TraceFix uses bounded retry, timeout-based execution, local-first operation, refusal for unsupported cases, escalation on weak evidence, and deterministic-first verification.

Accountability: Safety decisions are explicit and documented. Conservative behavior is the default, not an exception.

Escalation and Human-in-the-Loop Procedures

TraceFix escalates to human review when:

1. The patched script runs but no expected output is available.
2. The patch is too broad or behavior cannot be verified.

3. The diagnosis has low confidence or multiple plausible repair paths.
4. The system reaches a point where automatic acceptance would require guessing user intent.

Human review is supported by the following artifacts:

Artifact	Purpose	Key Information
trace.jsonl	Complete event timeline	Handoffs, component outputs, decisions, timestamps
session_state.json	Structured state	Session metadata, attempts, final decision, behavior match status
summary.md	Human-readable summary	What happened, why, and final outcome
patch files and diffs	Intermediate artifacts	Candidate changes and localized edits
failure_summary.md	Failure or escalation explanation	Trigger, rationale, and next step

Governance Summary

TraceFix implements governance through scope control, state control, evidence control, and conservative verification. The system is designed to avoid false-positive acceptance, prevent endless retry loops, document unresolved cases, and escalate when evidence is insufficient.

This framework makes TraceFix appropriate for its intended setting: a course project and portfolio-ready agentic system that demonstrates meaningful component coordination while staying honest about its boundaries.

8. Lessons Learned and Future Improvements

This project showed that TraceFix's most useful design choice is the separation between execution, diagnosis, patching, and verification. Debugging is not only a code-generation task. A patch that looks reasonable may still be wrong, and a script that no longer crashes may still fail to preserve the intended behavior. By keeping the Verifier separate from the Patcher, TraceFix can reject superficial fixes, stop on unsupported cases, and escalate when evidence is insufficient.

Another effective design choice was the controller-centered architecture. The Controller owns the session lifecycle, retry budget, state, and artifact persistence, while the Executor, Diagnoser, Patcher, and Verifier remain focused on their own responsibilities. This made the system easier to test and easier to explain. It also made the final evidence package more useful because each decision can be traced through trace.jsonl, session_state.json, summaries, patch diffs, and verifier outputs.

The conservative patching strategy also worked well for the current scope. For simple syntax errors, variable typos, argument mismatches, and missing-file guards, TraceFix was able to produce localized fixes and verify them with expected output. The system behaved especially well in cases where a broader or more speculative repair would have been risky. Stopping or escalating in those cases made the system less automatic, but more trustworthy.

Through this project, our team also learned how the course concepts translate into a real agentic system. Earlier in the course, we learned that an agentic system is not just a model response, but a system that perceives, reasons, acts, and continues through a task-oriented loop. TraceFix helped us apply that idea in a concrete debugging workflow: the Executor observes program behavior, the Diagnoser interprets the failure, the Patcher acts by proposing a repair, and the Verifier checks whether the action should be trusted. This made the perceive-think-act pattern more practical than abstract.

The project also reinforced the importance of bounded agent design. A good agentic system should start with a clear goal, known inputs and outputs, limited permissions, measurable success criteria, and visible escalation conditions. TraceFix forced us to make those decisions explicitly. We had to decide what the system was allowed to repair, when it should refuse, when it should ask for human review, and what evidence was strong enough for acceptance. This changed how we think about agentic complexity: more agents or components are only useful when they create clearer responsibility, better evidence, or safer decisions.

Another major lesson was the importance of state and memory design. TraceFix does not need long-term user memory, but it does need session state, retry history, verifier feedback, and artifact persistence. This helped us understand that memory in agentic systems is not only about personalization. It is also about continuity, auditability, and preventing repeated failed actions. The final design therefore uses stateless worker components and a stateful Controller, which keeps memory centralized and easier to inspect.

At the same time, the project revealed several limitations. First, TraceFix's repair coverage is still narrow. It handles a small set of beginner-to-intermediate bug classes,

but it does not yet support broader logic bugs, multi-file errors, dependency problems, or complex runtime failures. The unsupported IndexError case shows this boundary clearly. The system could identify the failure, but it could not safely infer the intended fix.

Second, verification is strongest only when expected output is available. The no-oracle case showed that successful execution alone is not enough evidence for acceptance. This is a good governance choice, but it also limits automation. In future versions, TraceFix should support more flexible behavior oracles, such as simple assertions, property checks, unit tests, or user confirmation prompts.

Third, the current evaluation is useful but still limited. The seven-case evaluation set covers the main workflow and key governance failures, but it is still small and curated. A stronger evaluation would include more diverse scripts, more bug classes, repeated runs, comparison against stronger baselines, and tests with real beginner-written code. It would also measure not only final decision accuracy, but also patch quality, explanation clarity, user trust, and how useful the artifacts are during human review.

Future work should improve TraceFix in four directions. First, the Patcher should support more safe bounded repair strategies, such as limited index guards, simple string-format corrections, and more robust file-handling patterns. Second, the Verifier should support richer behavior checks beyond exact expected-output matching. Third, the frontend should make traces, diffs, verifier decisions, and failure summaries easier for users to inspect. Fourth, the evaluation suite should be expanded into a larger benchmark with more realistic debugging tasks and stronger baseline comparisons.

Overall, the main lesson is that agentic debugging systems should not optimize only for producing patches. They should optimize for evidence-supported repair decisions. TraceFix works best when it treats debugging as a controlled workflow: collect evidence, diagnose the failure, propose a minimal patch, verify behavior, and stop or escalate when confidence is not strong enough. Future improvements should preserve this conservative structure while expanding the range of bugs and evidence sources the system can handle.

9. Individual Contributions

Phase 3 work continued from the Phase 2 division of responsibilities. The team kept the same general ownership structure across architecture, coordination and data design, prototype implementation, evaluation, governance, and final documentation. In this final phase, the main focus shifted from prototype design to final evidence collection, failure analysis, refinement, and portfolio-quality packaging.

Fan Yang continued to lead the architecture section. This work included updating the system architecture diagram, refining the role definitions, and making sure the final report clearly explained how the Controller, Executor, Diagnoser, Patcher, and Verifier interact. Fan also helped align the architecture description with the final implementation, including the retry loop, outcome states, human review points, and artifact flow.

Siru Tao was responsible for coordination logic, state and data design, and final report integration. This included documenting the handoff sequence, retry flow, stopping conditions, component interfaces, session state structure, artifact structure, and write permissions. Siru also helped connect the final report sections into a consistent narrative, especially the architecture, implementation summary, lessons learned, and individual contribution sections.

Xin Xu led the prototype implementation and final system refinement. This included building and maintaining the Controller, Executor, Diagnoser, Patcher, Verifier, CLI, sandbox policy, evaluation runner, visual API, and local frontend support. Xin also helped ensure that the project could run end-to-end, produce session artifacts, save patch outputs, and support the final evaluation run.

Crystal Huang led the evaluation, failure analysis, and governance work. This included preparing the evaluation cases, running or organizing the final evaluation evidence, documenting the actual results, preparing the failure log, and refining the governance and safety discussion. Crystal also helped ensure that the evaluation package included realistic repairable cases, unsupported cases, superficial-fix cases, and no-oracle escalation cases.

As a team, we completed four main outcomes in Phase 3. First, we finalized TraceFix as a runnable Track A system with a working debugging workflow from execution to diagnosis, patching, rerun, and verification. Second, we completed the evaluation package with seven executed test cases, actual results, failure cases, and saved artifacts. Third, we documented the main failure modes, governance controls, sandbox boundaries, and escalation behavior. Fourth, we prepared the final project package, including the report, README, evidence materials, screenshots, outputs, and video materials.

Overall, Phase 3 turned the Phase 2 prototype into a final, reviewable project package. Each team member continued to own a clear part of the project, while the final submission required shared review across implementation, evaluation, documentation, and presentation.

Appendix A. Screenshot Index

Figure A1. Home landing screen.

Caption: TraceFix frontend landing screen showing the project purpose, local visual demo layer, component sequence, target user, and scoped debugging workflow.

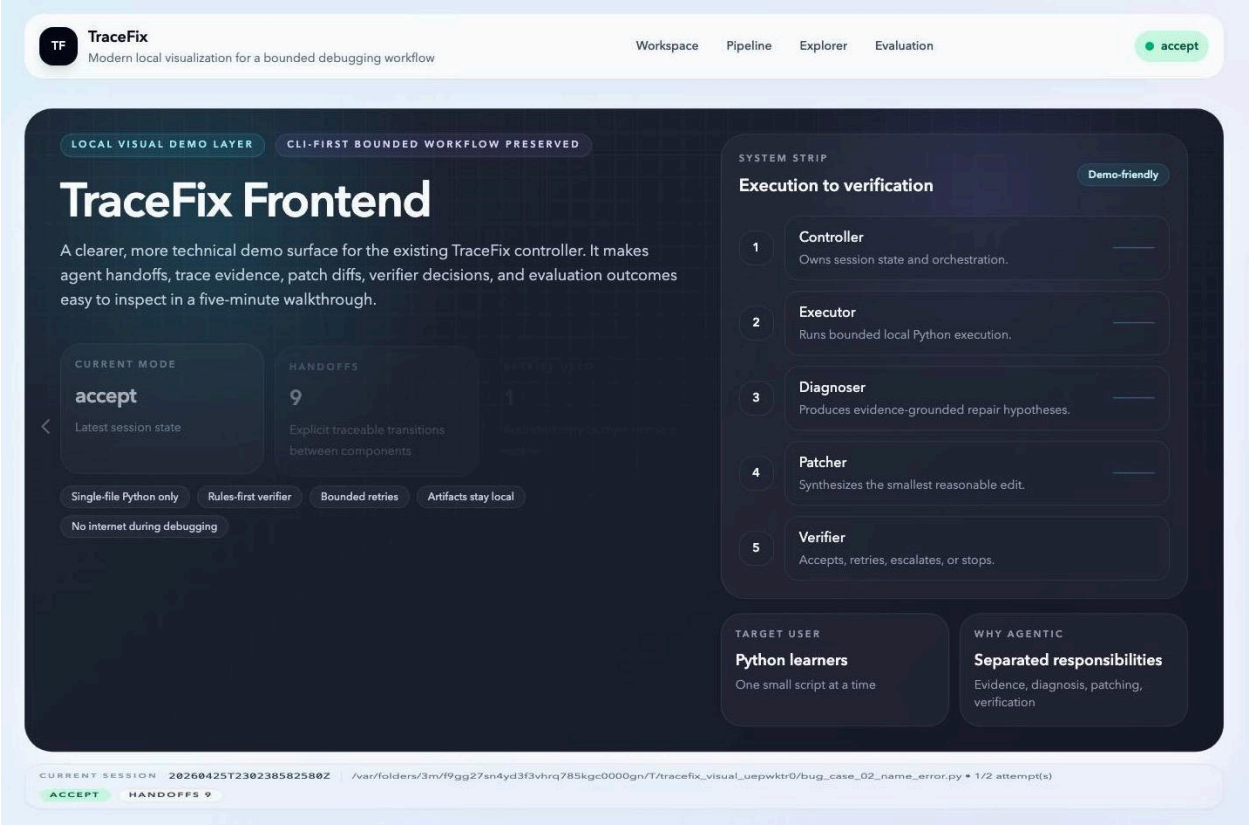


Figure A2. Main interaction screen.

Caption: Main workspace where users select a sample case, inspect or paste a single-file Python script, provide expected output, set retry limits, and run TraceFix.

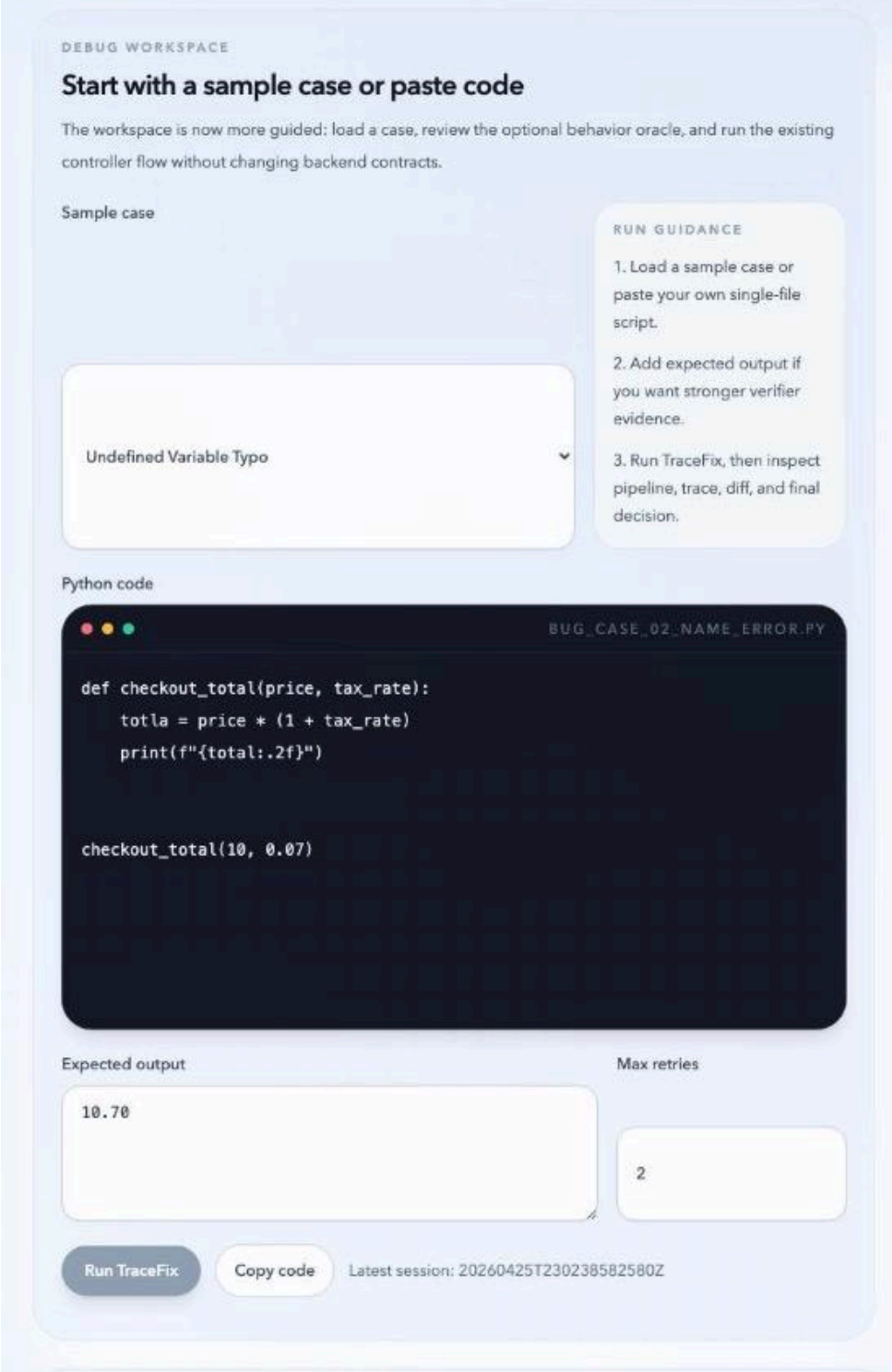


Figure A3. Demo case controls.

Caption: Quick-load demo scenario controls grouped by happy path, governance, and failure/stop cases, showing settings, controls, and scenario selection for walkthrough and evaluation review.

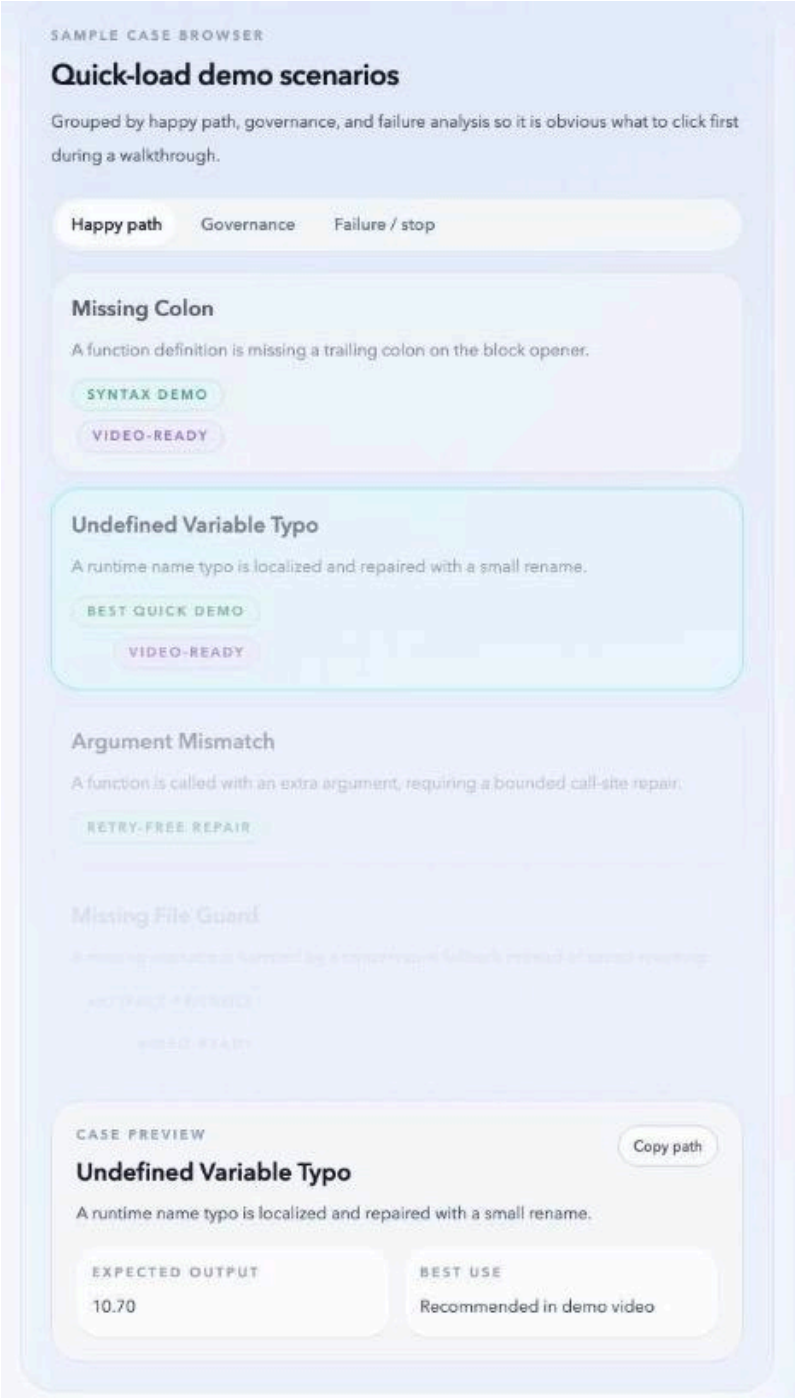


Figure A4. Component handoff and decision view.

Caption: Component handoff and final decision view showing Controller, Executor, Diagnoser, Patcher, and Verifier outputs for an accepted repair case.

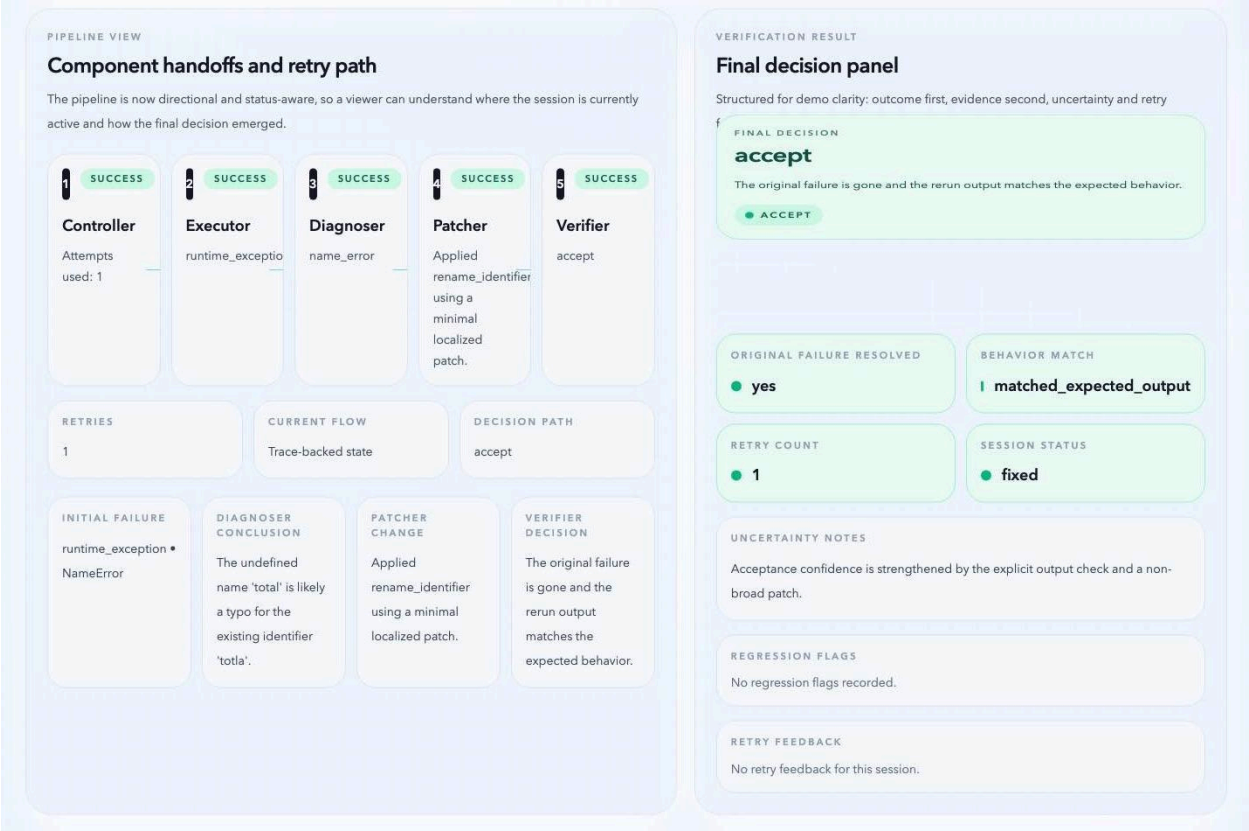


Figure A5. Trace, diff, and artifact explorer.

Caption: Session explorer showing trace timeline, patch diff, patch attempts, and artifact tabs used as the evidence layer for post-hoc review.

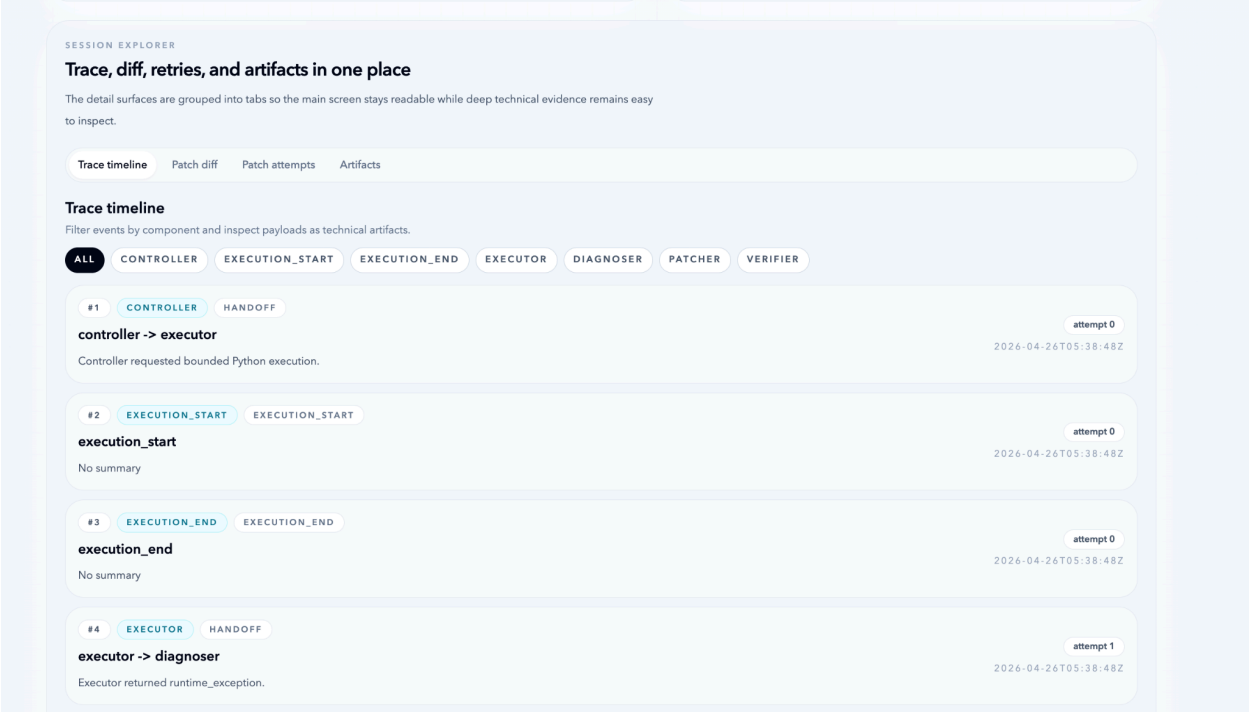


Figure A6. Evaluation and failure dashboard.

Caption: Evaluation dashboard showing total cases, accepted cases, stopped cases, escalated cases, outcome distribution, and two high-value limitation stories.

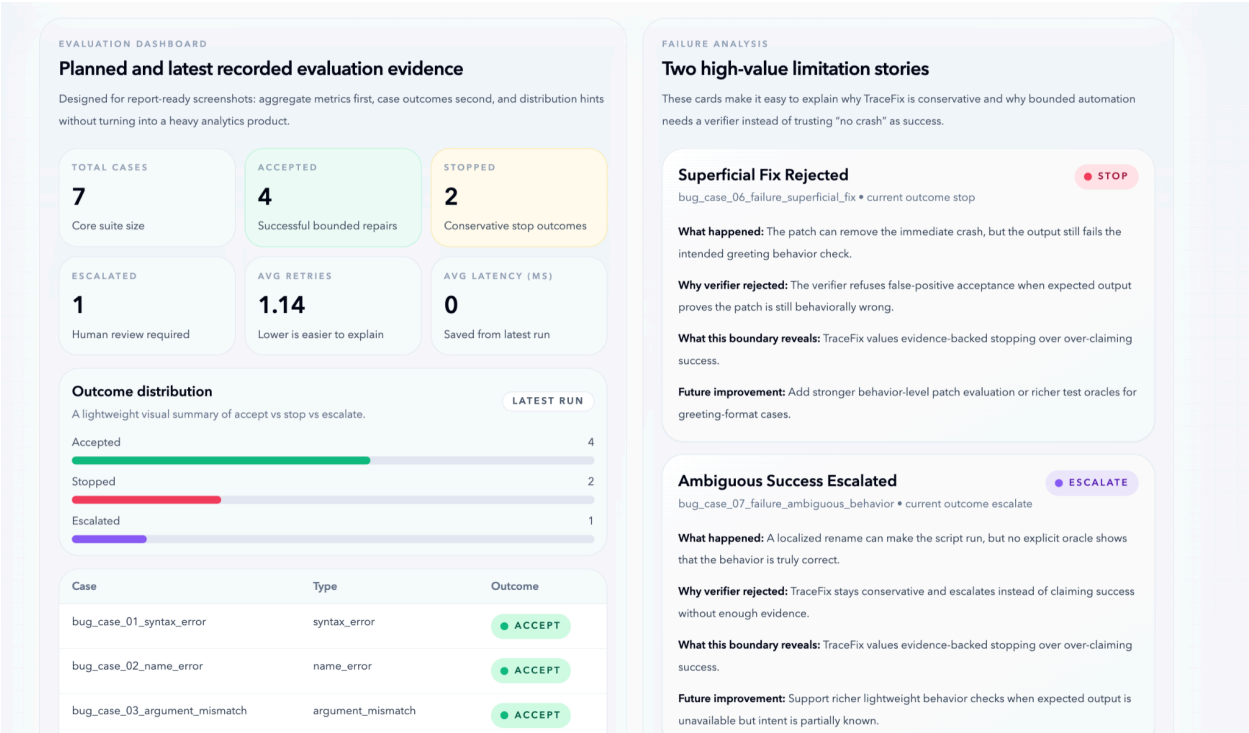
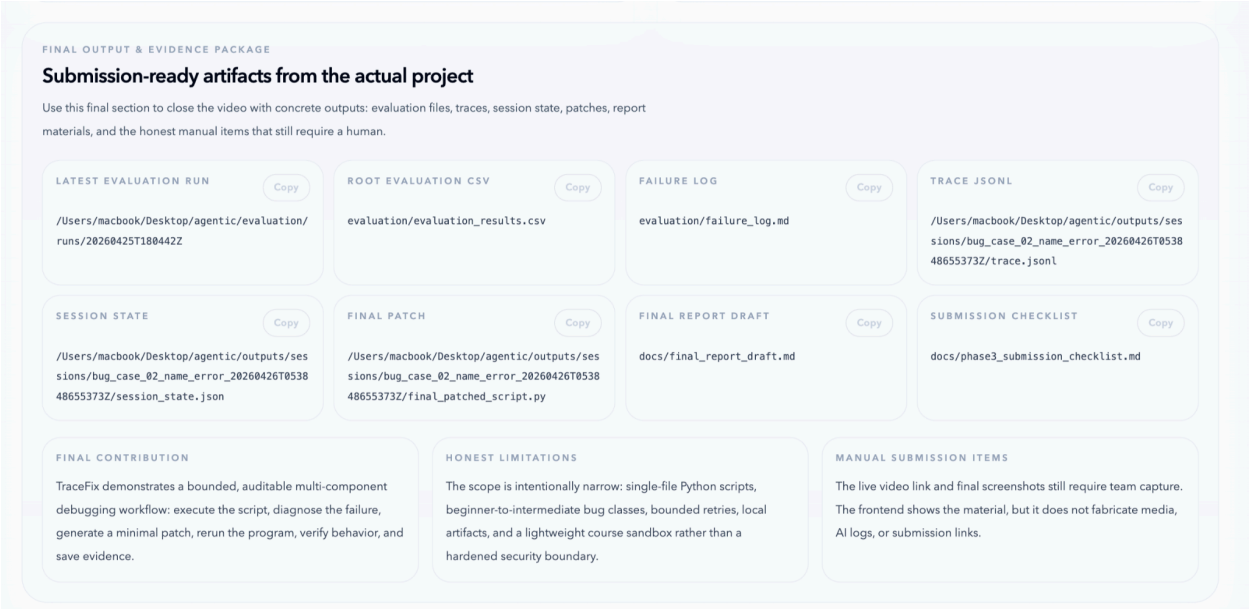


Figure A7. Final output and evidence package.

Caption: Final output and evidence package view showing the latest evaluation run, evaluation results CSV, failure log, trace JSONL, session state, final patch, report draft, and submission checklist.